

PROGRAMAÇÃO DE DISPOSITIVOS MÓVEIS

Prof. Tiago Piperno Bonetti
bonetti@prof.unipar.br



Criando o *useReducer*

```
8 // estado inicial para a função reducer
9 // objeto com os usuários
10 const initialState = { users };
```

```
16 // função que representa o reducer
17 // recebe o estado e a uma ação como parâmetros de entrada
18 function reducer(state, action) {
19   // exibe a ação recebida
20   console.warn(action)
21   //retornando o estado atualizado
22   return state
23 }
24
25 //useReducer recebendo a função criada como parâmetro
26 //segundo parâmetro: estado dos usuários
27 //vamos receber como resposta duas informações:
28 //o estado e o dispatch (responsável por disparar os eventos)
29 const [state, dispatch] = useReducer(reducer, initialState);
```

```
32 <UserContext.Provider
33   value={{
34     //substituímos o state anterior pelo do reducer
35     //constinuamos acessando os dados, agora regerenciados pelo useReducer
36     //passamos também o dispatch (forma de invocar um evento)
37     state, dispatch
38   }}>
```

Usando o *useReducer*

```
9 //recebendo o dispatch via contexto
10 const { state, dispatch } = useContext(UsersContext)
```

```
28 //função para verificar a exclusão
29 function confirmUserDeletion(user) {
30   Alert.alert('Excluir Usuário', 'Deseja realmente excluir usuário', [
31     {
32       text: 'Sim',
33       onPress() {
34         //quando clicar em sim vamos disparar uma action
35         //uma action é um objeto
36         //a ação vai gerar uma nova versão do estado(lista de usuários)
37         dispatch({
38           type: 'deleteUser',
39           payload: user, //dado que está sendo passado para a ação
40         })
41       },
42     },
```

Evoluindo o estado - Exclusão

```
13 export const UsersProvider = (props) => {
14   // função que representa o reducer
15   // recebe o estado e a uma ação como parâmetros de entrada
16   function reducer(state, action) {
17
18     //se receber uma action desse tipo
19     if (action.type == 'deleteUser') {
20       //recebe o usuário
21       const user = action.payload;
22       return {
23         //clonando todos os atributos atuais do estado
24         ...state,
25         //usa a função filter para excluir o usuário
26         //filtra o usuário com id igual ao recebido
27         //se for diferente, permanece na lista
28         users: state.users.filter((u) => u.id !== user.id),
29       };
30     }
31   }
```

Evoluindo o estado - Inclusão

```
32 // função para criar um usuário
33 if (action.type == 'createUser') {
34   const user = action.payload;
35   // pegando o id do último usuário e somando 1
36   user.id = [...state.users].pop().id + 1;
37
38   return {
39     ...state,
40     //adicionando usuário a lista
41     users: [...state.users, user],
42   };
43 }
```

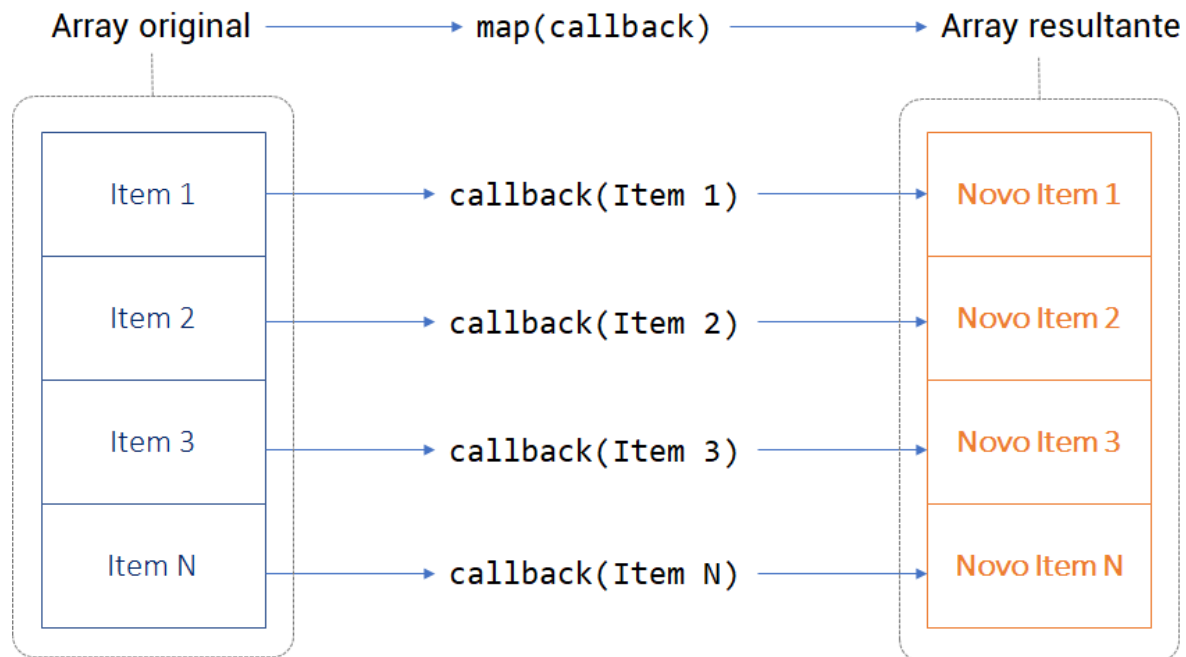
Evoluindo o estado - Inclusão

```
1  import React, { useState, useContext } from 'react';
2  import { Text, View, TextInput, StyleSheet, Button } from 'react-native';
3  import UsersContext from '../context/UsersContext';
4
5  export default (props) => {
6    //acessando apenas o dispatch
7    const { dispatch } = useContext(UsersContext)
```

```
43    <Button
44      title="Salvar"
45      onPress={() => {
46        dispatch({
47          //verifica se tem id
48          type: user.id ? 'updateUser' : 'createUser',
49          payload: user,
50        });
51        props.navigation.goBack();
52      }}
53    />
```

Função `map()`

O método `map()` invoca a função `callback` passada por argumento para cada elemento do `Array` e **devolve um novo `Array` como resultado**.



Função map()

```
const lista = [2, 4, 6, 8];  
const multiplicados = lista.map( (numero) => numero * 2);  
console.log(multiplicados);
```

```
Chrome: ▾ [ 4, 8, 12, 16 ]  
  0: 4  
  1: 8  
  2: 12  
  3: 16  
 length: 4
```

```
const lista2 = [2, 4, 3, 9];  
const multiplicaPar = lista2.map( (numero) =>  
  | numero % 2 == 0 ? numero * 2 : numero  
  );
```

```
Chrome: ▾ [ 4, 8, 3, 9 ]  
  0: 4  
  1: 8  
  2: 3  
  3: 9  
 length: 4
```


Evoluindo o estado - Alteração

```
45   if (action.type == 'updateUser') {  
46     const userUpdate = action.payload;  
47  
48     return {  
49       ...state,  
50       //gerando outro array com o usuário atualizado  
51       //se tiver um id igual o do que foi atualizado, retorna os novos dados  
52       users: state.users.map( (user) => user.id === userUpdate.id ? userUpdate : user  
53     ),  
54   };  
55 }
```

Referências

Construa um Cadastro COMPLETO em React Native - Com Hooks e Context API: <https://youtu.be/V-uYjDnuXkU>

Escudelario, Bruna de Freitas; Pinho, Diego Martins. **React Native: Desenvolvimento de aplicativos mobile com React**. Casa do código.

Documentação React Native. Disponível em: <https://reactnative.dev/docs>.

Documentação React Navigation. Disponível em: <https://reactnavigation.org/docs/getting-started>

<https://oblador.github.io/react-native-vector-icons/>